Nama : Muhammad Syafan Midhad

NIM : A11.2023.14923

Hasil Latihan Optimisasi DB

KODE SUMBER

Course + Instructor

Baseline

```
def course_list_baseline(request):
    courses = Course.objects.all()
    data = []

    for c in courses:
        data.append({
            'course': c.title,
            'instructor': c.instructor.username,
        })

    return JsonResponse({'data': data})
```

Optimize

```
def course_list_optimized(request):
    courses = Course.objects.select_related('instructor').all()
    data = []

    for c in courses:
        data.append({
            'course': c.title,
            'instructor': c.instructor.username,
        })

    return JsonResponse({'data': data})
```

Course + Members

Baseline

```
def course_members_baseline(request):
    courses = Course.objects.all()
    payload = []

    for c in courses:
        member_count = Enrollment.objects.filter(course=c).count()
        payload.append({
            'course': c.title,
            'member_count': member_count,
        })

    return JsonResponse({'data': payload})
```

Optimized

```python
def course_members_optimized(request):
    courses = Course.objects.annotate(
        member_count=Count('enrollment')
    )

    payload = []

    for c in courses:
        payload.append({
            'course': c.title,
            'member_count': c.member_count,
        })

    return JsonResponse({'data': payload})
```

Dashboard Statistik

Baseline

```python
def course_dashboard_baseline(request):
    courses = Course.objects.all()

    total_courses = courses.count()

    total_enrollments = 0
    for c in courses:
        total_enrollments += Enrollment.objects.filter(course=c).count()

    avg_enrollment = total_enrollments / total_courses if total_courses > 0 else 0

    return JsonResponse({
        'total_courses': total_courses,
        'total_enrollments': total_enrollments,
        'avg_enrollment': avg_enrollment,
    })
```

Optimized

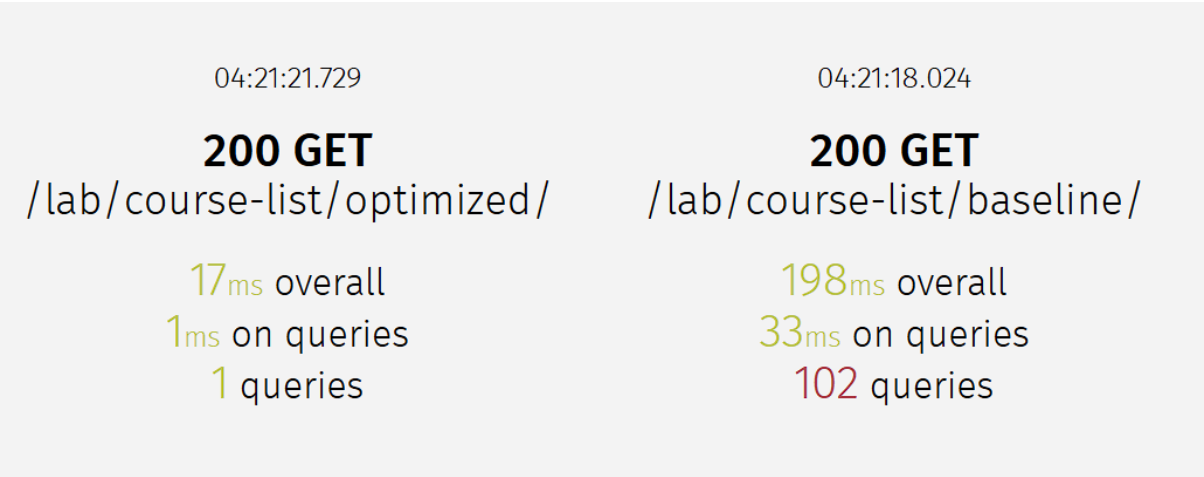
```python
def course_dashboard_optimized(request):
    stats = Course.objects.annotate(
        member_count=Count('enrollment')
    ).aggregate(
        total_courses=Count('id'),
        total_enrollments=Count('enrollment'),
        avg_enrollment=Avg('member_count'),
    )

    return JsonResponse(stats)
```

TABEL PERBANDINGAN SILK

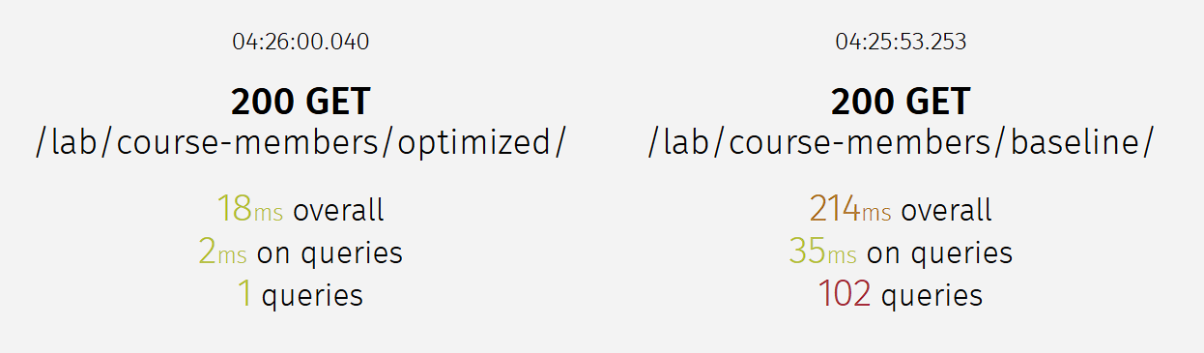
Kasus	Endpoint Baseline	Endpoint Optimized	Perbandingan Silk	Teknik
Course + teacher	/lab/course-list/baseline/	/lab/course-list/optimized/	102 query → 1 query, 198ms → 17ms	select_related
Course + members	/lab/course-members/baseline/	/lab/course-members/optimized/	102 query → 1 query, 214ms → 18ms	prefetch_related
Statistik dashboard	/lab/course-dashboard/baseline/	/lab/course-dashboard/optimized/	103 query → 1 query, 201ms → 17ms	aggregate/annotate

SCREENSHOT SILK (sebelum & sesudah)

Course-list



Course-members



Course-dashboard

04:26:58.627	04:26:56.495
200 GET /lab/course-dashboard/optimized/	**200 GET** /lab/course-dashboard/baseline/
17ms overall	201ms overall
2ms on queries	30ms on queries
1 queries	103 queries

RINGKASAN TEKNIK & DAMPAK

1. select_related

Digunakan untuk mengambil relasi ForeignKey dalam satu query menggunakan JOIN.
Dampak: menghilangkan N+1 problem dan menurunkan query dari ratusan menjadi 1.

2. annotate

Digunakan untuk menghitung agregasi (seperti jumlah member) langsung di database.
Dampak: menghindari query berulang dalam loop dan meningkatkan efisiensi.

3. aggregate

Digunakan untuk menghitung statistik global seperti total dan rata-rata dalam satu query.
Dampak: mengurangi beban komputasi di Python dan mempercepat respon.

KESIMPULAN

Optimasi query menggunakan select_related, annotate, dan aggregate berhasil mengatasi N+1 problem yang terjadi pada baseline. Jumlah query berhasil dikurangi secara signifikan (hingga lebih dari 90%) dan waktu respon meningkat drastis. Hal ini menunjukkan bahwa optimasi query pada ORM Django sangat penting untuk meningkatkan performa aplikasi.